

RELATÓRIO DO TRABALHO 1

COMUNICAÇÃO ENTRE PROCESSOS

Universidade Federal do Ceará – Campus Quixadá

Disciplina: Sistemas Distribuídos

Professor: Antonio Rafael Braga

Serviço remoto escolhido:

Sistema de gerenciamento de biblioteca (Extra: Sistema distribuído de votação).

Identificação dos Estudantes

Nome completo	Matrícula	Curso
Milena Helen Diniz Gomes	565747	Ciência da Computação
Yasmin de Lima Marques	567615	Ciência da Computação

Aprendizados e Dificuldades

Durante a realização da atividade, conseguimos compreender melhor como funciona a comunicação entre processos em sistemas distribuídos, utilizando sockets para estabelecer a comunicação entre cliente e servidor. Também aprendemos na prática o uso de TCP para troca de informações, UDP multicast para envio de mensagens para múltiplos clientes, além da criação de streams personalizados e da serialização e desserialização de objetos. Outro aprendizado importante foi entender o papel das threads no funcionamento de aplicações distribuídas e a importância de organizar corretamente os componentes de um sistema desse tipo.

Entre as principais dificuldades encontradas, tivemos a configuração e o entendimento da comunicação multicast utilizando UDP, principalmente por envolver grupos multicast, interfaces de rede e a necessidade de garantir que as mensagens fossem enviadas e recebidas corretamente.

Também encontramos desafios na implementação das streams personalizadas e na organização do processo de envio e recuperação dos dados. Além disso, o uso de threads para controlar diferentes tarefas simultaneamente exigiu mais atenção durante a implementação.

Avaliação da Atividade

Nota atribuída pela dupla

10 / 10

Nível de dificuldade

☐ Muito fácil ☐ Fácil ☐ Média ☒ Difícil ☐ Muito difícil

Tempo gasto na atividade

9 horas

Repositório do Código Fonte

Link do GitHub: [Repositório](#)

Descrição da Implementação

A aplicação desenvolvida consiste em um sistema distribuído de gerenciamento de biblioteca, utilizando diferentes formas de comunicação entre processos. Foram implementadas funcionalidades para gerenciamento de livros e empréstimos utilizando sockets TCP para comunicação entre cliente e servidor, streams personalizados para envio e recebimento de objetos e serialização para troca de informações entre aplicações. Além disso, foi implementada uma comunicação multicast utilizando UDP para envio de notificações aos clientes conectados. Também foram utilizadas threads para permitir o funcionamento simultâneo das operações e o atendimento de múltiplas conexões.

Implementação utilizando Sockets e Streams

Classes POJO desenvolvidas

Foram implementadas classes POJO responsáveis por representar as entidades utilizadas no serviço remoto, permitindo organizar os dados trocados entre cliente e servidor durante a comunicação. Essas classes também foram utilizadas como estruturas de dados para representar as informações trocadas durante a comunicação entre cliente e servidor.

Classe	Descrição
Livro	Representa a entidade principal do sistema de biblioteca, armazenando informações dos livros como ISBN, título, autor, ano de publicação e disponibilidade.
Requisicao	Representa as solicitações enviadas pelo cliente ao servidor, contendo as informações necessárias para identificar a operação que deve ser realizada.
Resposta	Representa as respostas retornadas pelo servidor ao cliente após o processamento das requisições, contendo os dados ou mensagens resultantes da operação executada.
TipoOperacao	Define os tipos de operações disponíveis no sistema, permitindo identificar qual ação deve ser executada pelo servidor a partir da requisição recebida.

Implementação do OutputStream personalizado

LivroOutputStream

Descrição:

A classe **LivroOutputStream** foi implementada para realizar a escrita dos dados de objetos **Livro** em um fluxo de saída personalizado. Ela permite transformar os atributos do objeto em uma sequência de dados que pode ser armazenada ou transmitida através dos fluxos utilizados pela aplicação. Foram realizados testes para validar o funcionamento da implementação, verificando o envio dos objetos pela entrada e saída padrão e a gravação dos dados em arquivo.

Evidências

Figura 1 – Execução do teste de escrita utilizando o LivroOutputStream para gravação dos objetos Livro.

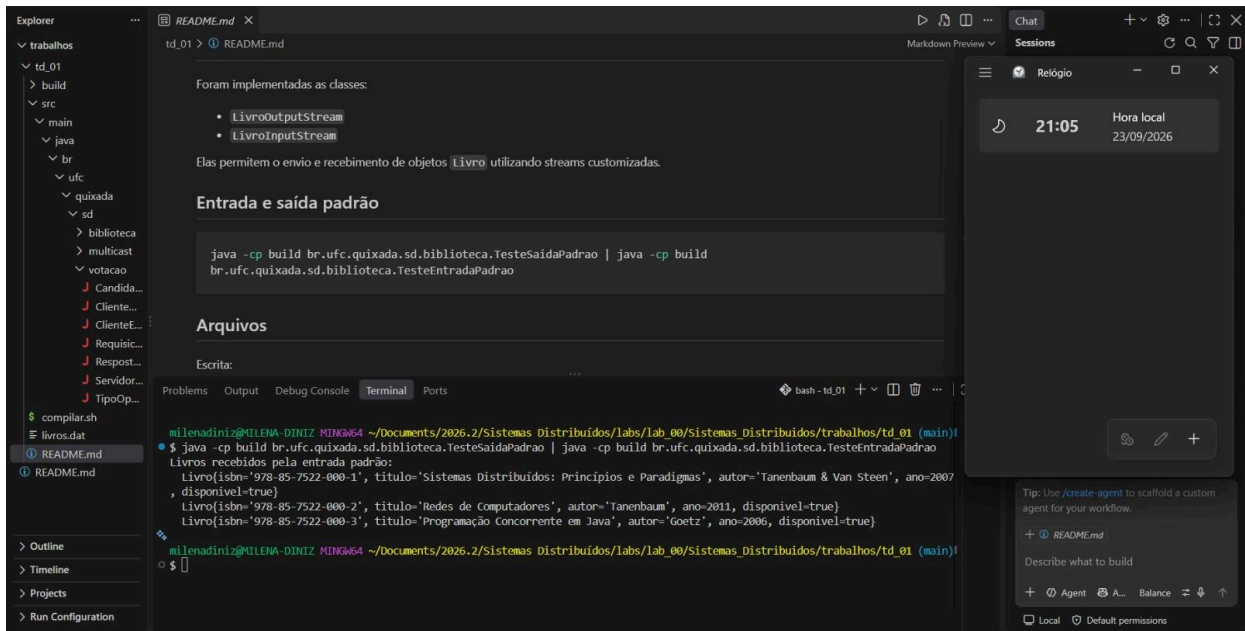
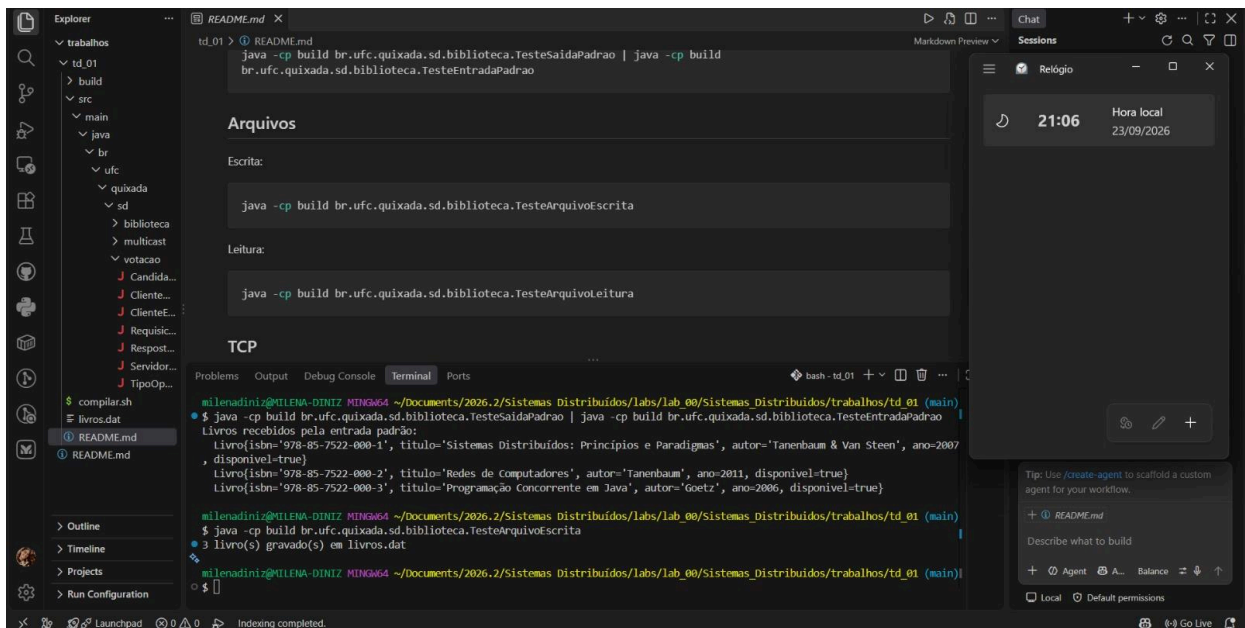


Figura 2 – Execução dos testes de escrita dos objetos Livro utilizando streams personalizados.



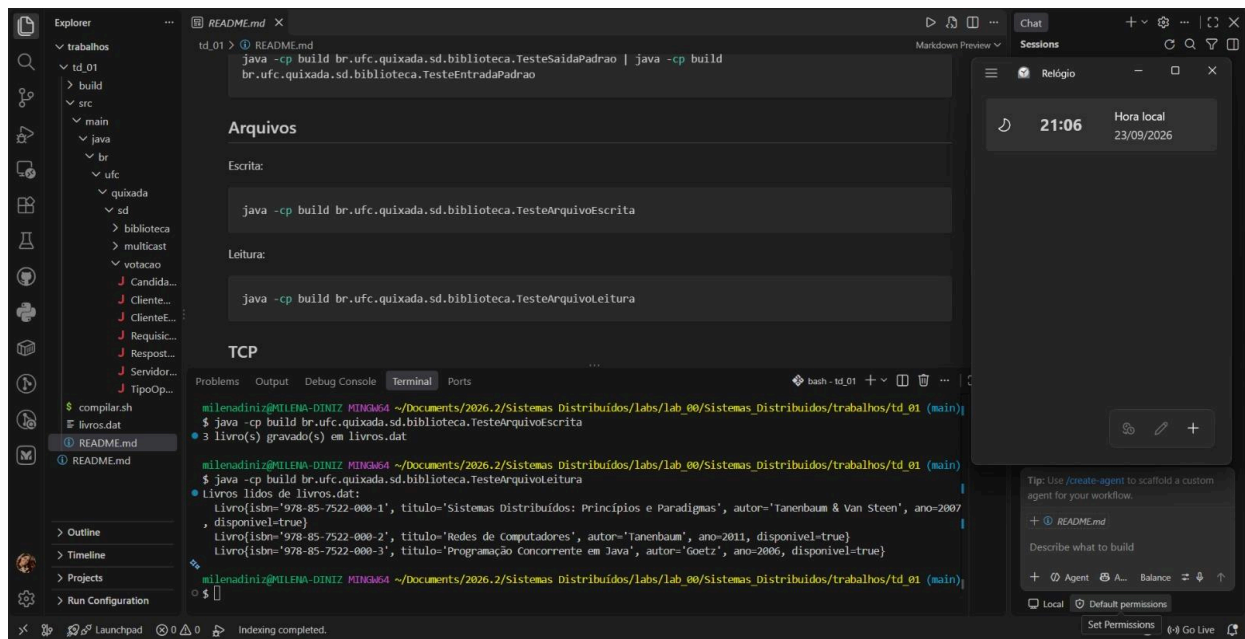
LivroInputSteam

Descrição:

A classe `LivroInputStream` foi implementada para realizar o processo inverso do `LivroOutputStream`, permitindo recuperar os objetos `Livro` a partir dos dados armazenados. A classe realiza a leitura das informações e reconstrói os objetos da aplicação. O teste utilizando arquivo confirmou que os dados armazenados poderiam ser lidos e convertidos novamente em objetos `Livro`.

Evidências

Figura 3 – Execução do teste de leitura utilizando o `LivroInputStream` para recuperar os objetos `Livro` armazenados.



Implementação utilizando Sockets TCP

Comunicação Cliente-Servidor

Foi implementada uma comunicação utilizando sockets TCP através das classes `ClienteTCPStream` e `ServidorTCPStream`. O servidor permanece aguardando conexões em uma porta específica e, ao receber uma solicitação do cliente, realiza o processamento da requisição e retorna os dados solicitados.

A comunicação foi realizada utilizando streams de objetos, permitindo o envio e recebimento das informações da aplicação através da serialização dos dados.

Fluxo da comunicação:

1. O cliente estabelece uma conexão TCP com o servidor;
2. O cliente envia uma requisição contendo a operação desejada;
3. O servidor recebe a solicitação e realiza o processamento;
4. O servidor retorna a resposta contendo os dados solicitados;
5. O cliente recebe e apresenta as informações retornadas.

Evidências

Figura 4 – Servidor TCP aguardando conexões e recebendo dados enviados pelo cliente.

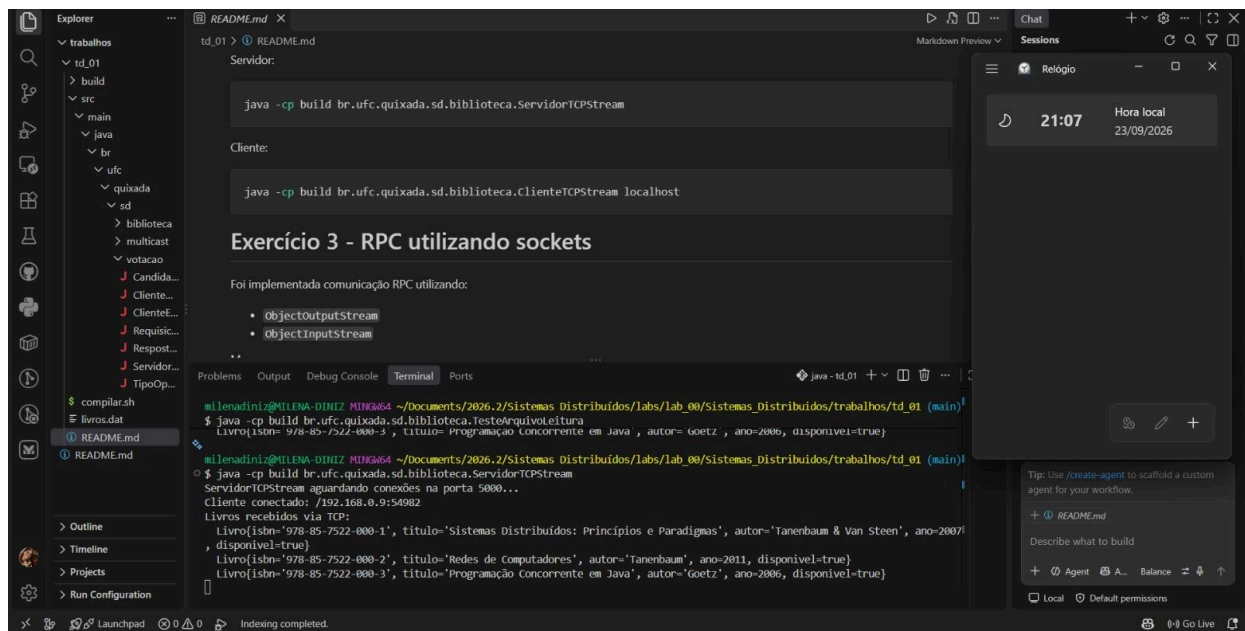
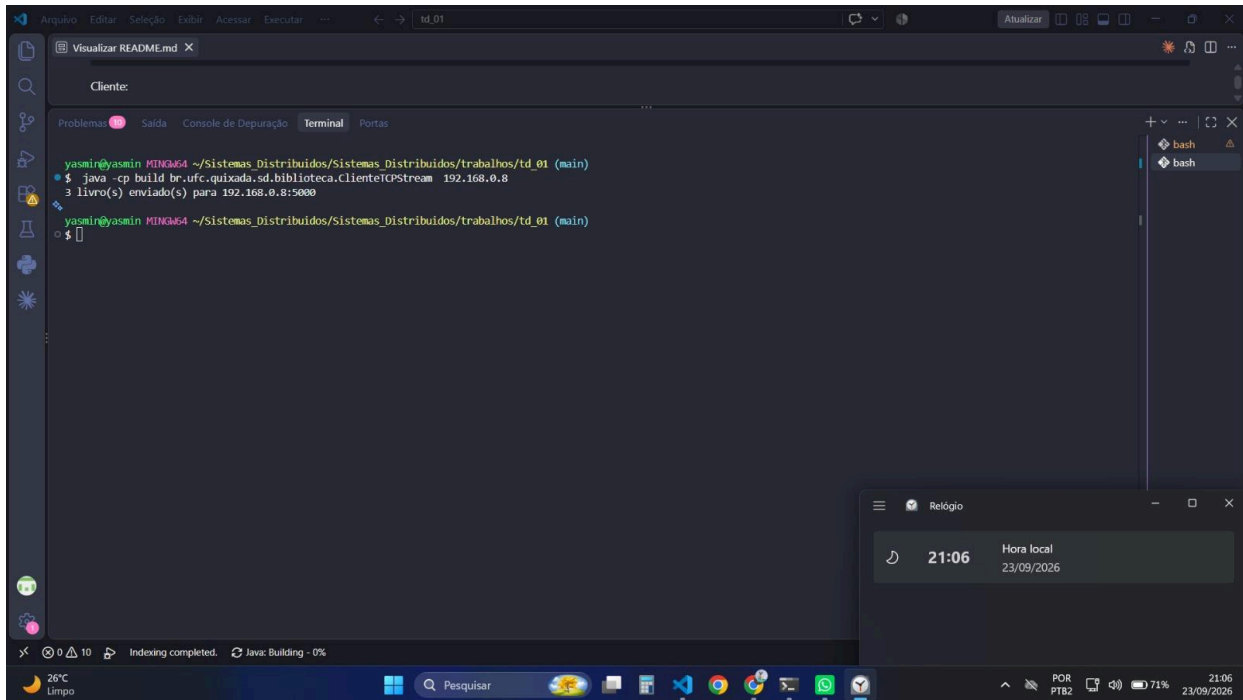


Figura 5 – Cliente TCP realizando a comunicação com o servidor através de sockets TCP.



Implementação de Serialização

Descrição

Foi implementada a serialização dos objetos utilizados no serviço remoto, permitindo que as informações fossem convertidas para um formato que permite o envio dos objetos através da comunicação estabelecida entre cliente e servidor.. Essa abordagem possibilitou o empacotamento dos dados antes do envio e a recuperação das informações recebidas através do processo de desserialização.

Na comunicação, o cliente realiza a criação da requisição, serializa o objeto e envia os dados através do socket. Ao receber a mensagem, o servidor realiza a desserialização das informações, processa a requisição, gera uma resposta e realiza novamente a serialização antes de enviá-la de volta ao cliente.

Evidências

Figura 6 – Execução do cliente realizando o envio de objetos Livro utilizando a comunicação baseada em streams.

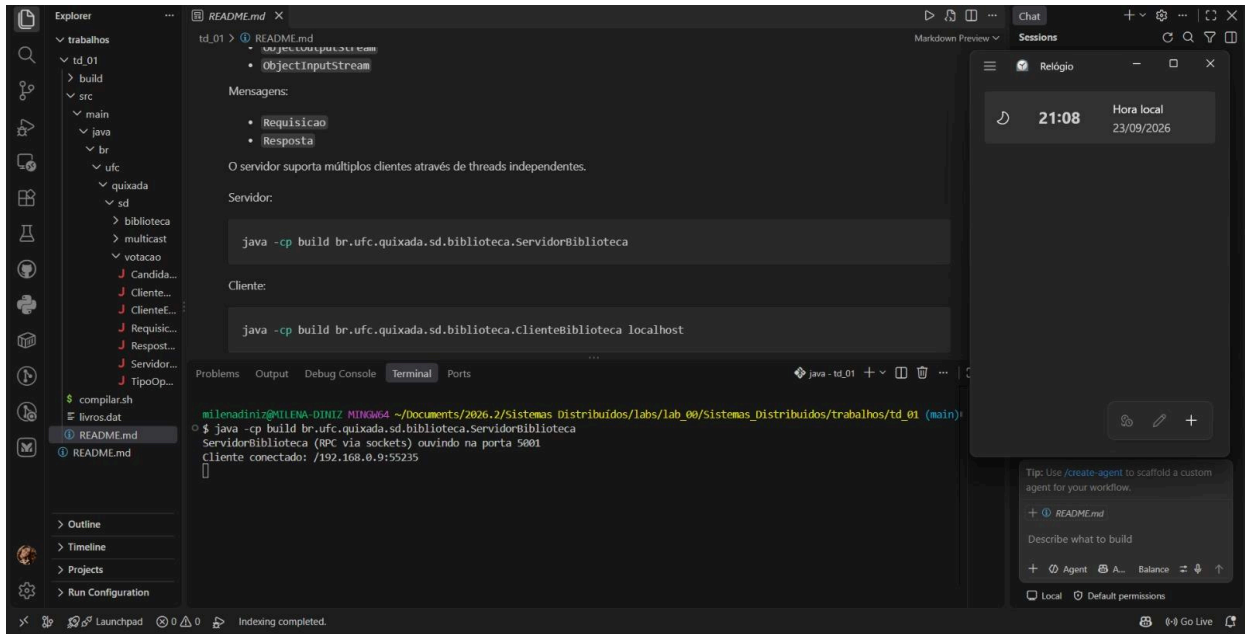
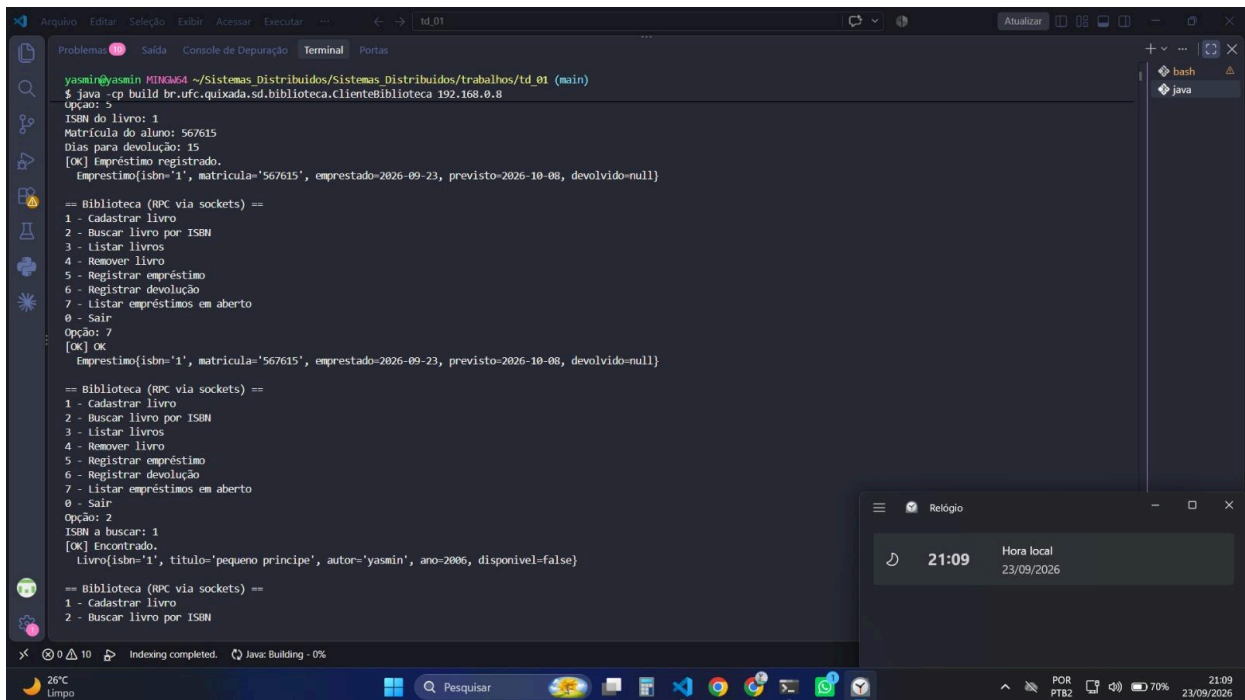


Figura 7 – Execução do servidor recebendo os objetos Livro enviados pelo cliente através da comunicação TCP.



Implementação Multicast

Descrição da implementação

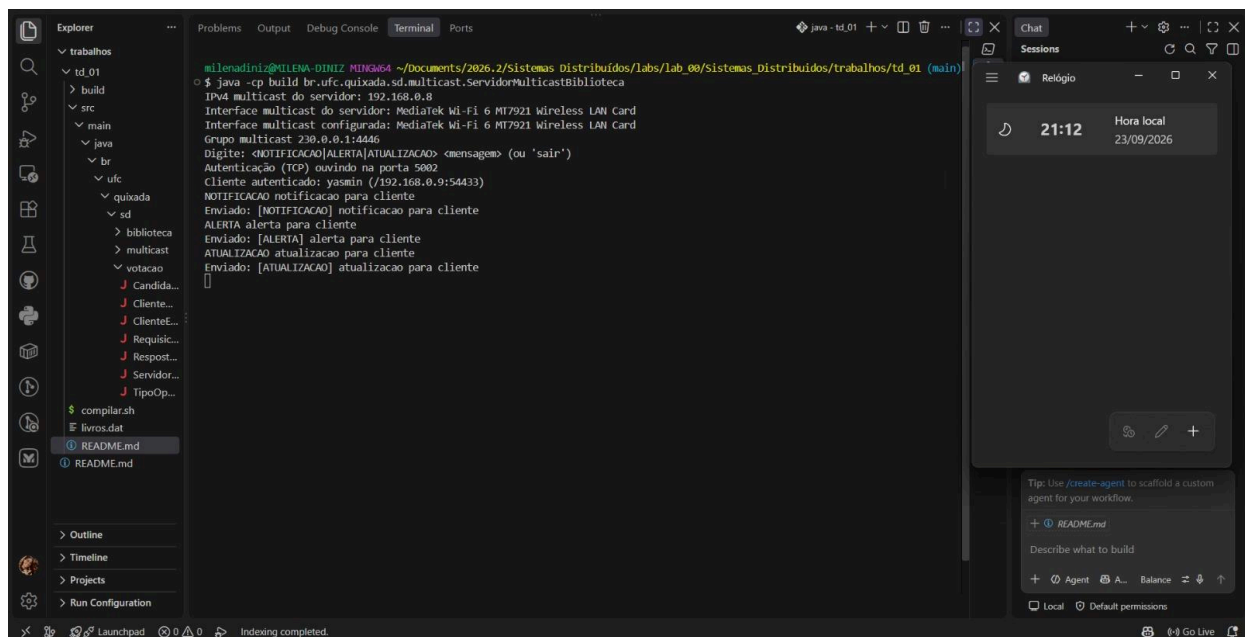
Foi implementada uma comunicação multicast utilizando UDP para permitir o envio de mensagens do servidor para múltiplos clientes conectados ao mesmo grupo. O servidor utiliza o endereço multicast **230.0.0.1** e a porta **4446** para realizar o envio das mensagens.

A aplicação permite o envio de diferentes tipos de notificações, como **NOTIFICACAO**, **ALERTA** e **ATUALIZACAO**, utilizando uma estrutura de dados contendo o tipo da mensagem, o conteúdo e o timestamp do envio. Os clientes conseguem entrar e sair do grupo multicast, permanecendo em escuta para receber as mensagens enviadas pelo servidor em tempo real.

Para permitir o funcionamento contínuo da aplicação, foi utilizada uma thread exclusiva para receber as mensagens multicast, enquanto a aplicação principal permanece disponível para interação com o usuário.

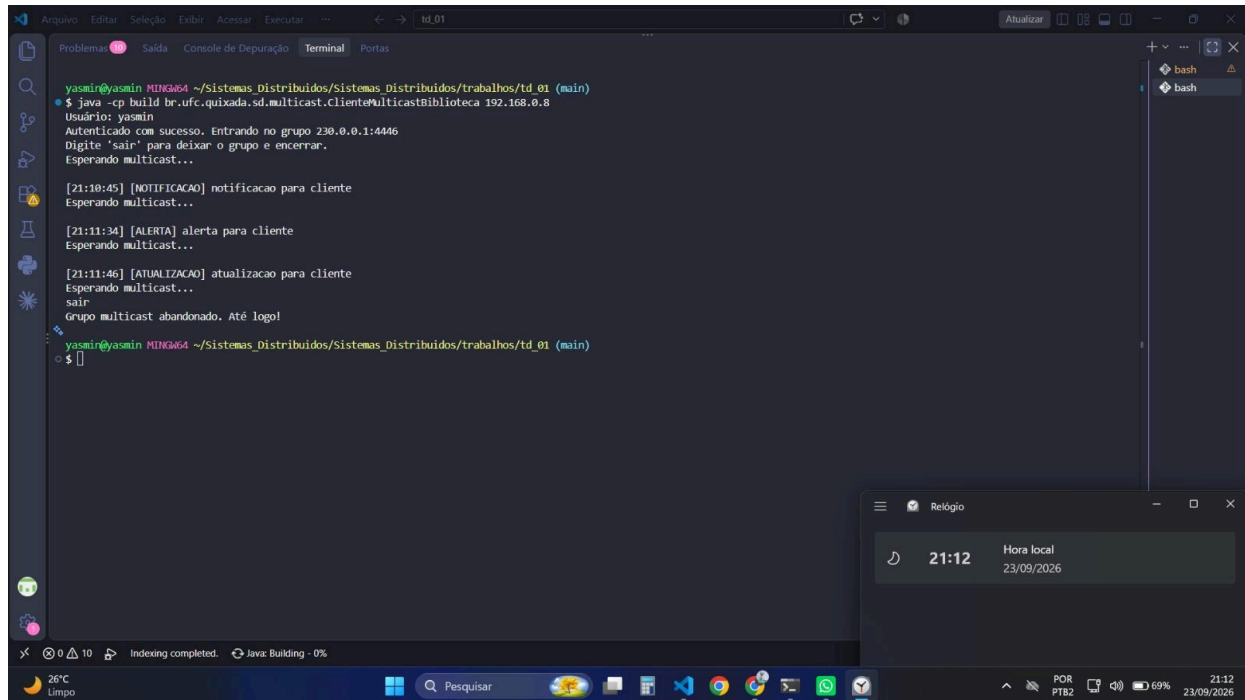
Evidências

Figura 8 – Execução do servidor multicast enviando mensagens de notificação para o grupo multicast.



```
milena@milena-DINIZ-MINGUEIRA: ~/Documents/2026.2/Sistemas Distribuídos/labs/lab_00/Sistemas_Distribuídos/trabalhos/td_01 (main)
$ java -cp build br.ufc.quixada.sd.multicast.ServidorMulticastBiblioteca
IPv4 multicast do servidor: 192.168.0.8
Interface multicast do servidor: MediaTek Wi-Fi 6 MT7921 Wireless LAN Card
Interface multicast configurada: MediaTek Wi-Fi 6 MT7921 Wireless LAN Card
Grupo multicast 230.0.0.1:4446
Digite: <NOTIFICACAO>[ALERTA][ATUALIZACAO] <mensagem> (ou 'sair')
Autenticação (TCP) ouvindo na porta 5002
Cliente autenticado: yassin (/192.168.0.9:54433)
NOTIFICACAO notificacao para cliente
Enviado: [NOTIFICACAO] notificacao para cliente
ALERTA alerta para cliente
Enviado: [ALERTA] alerta para cliente
ATUALIZACAO atualizacao para cliente
Enviado: [ATUALIZACAO] atualizacao para cliente
[]
```

Figura 9 – Recebimento das mensagens multicast pelo cliente através do grupo UDP configurado.



Questão Extra – Sistema de Votação Distribuído

Descrição da aplicação

A aplicação permite o gerenciamento de uma votação distribuída, incluindo autenticação de eleitores e administradores, listagem, cadastro e remoção de candidatos, registro de votos, consulta de resultados e envio de notificações aos usuários conectados.

A comunicação TCP foi utilizada para as operações que necessitam de uma comunicação direta entre cliente e servidor, como login, envio de informações da votação e registro dos votos. Já a comunicação multicast utilizando UDP foi utilizada para o envio de mensagens informativas pelos administradores aos eleitores conectados ao sistema. Além disso, foram utilizadas threads para permitir o atendimento de múltiplos clientes e o recebimento das notificações em tempo real.

Evidências

Figura 10 – Execução do servidor do sistema de votação realizando o processamento das operações solicitadas pelos clientes.

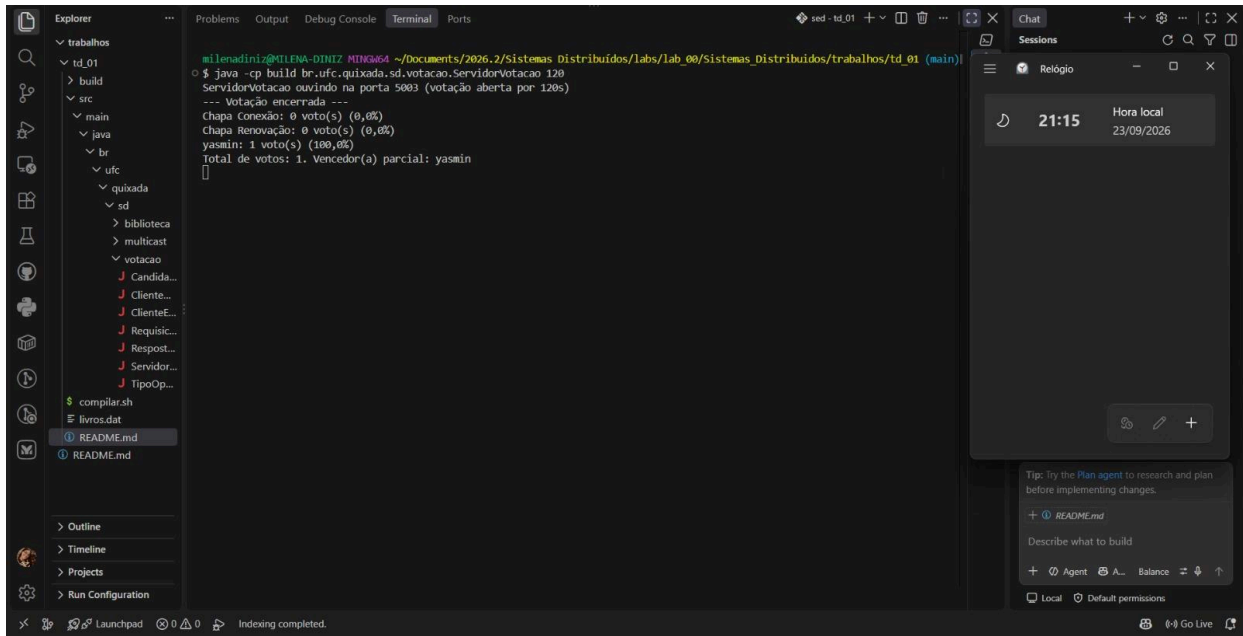


Figura 11 – Execução do cliente administrador realizando operações de gerenciamento da votação e envio de notificações aos eleitores.

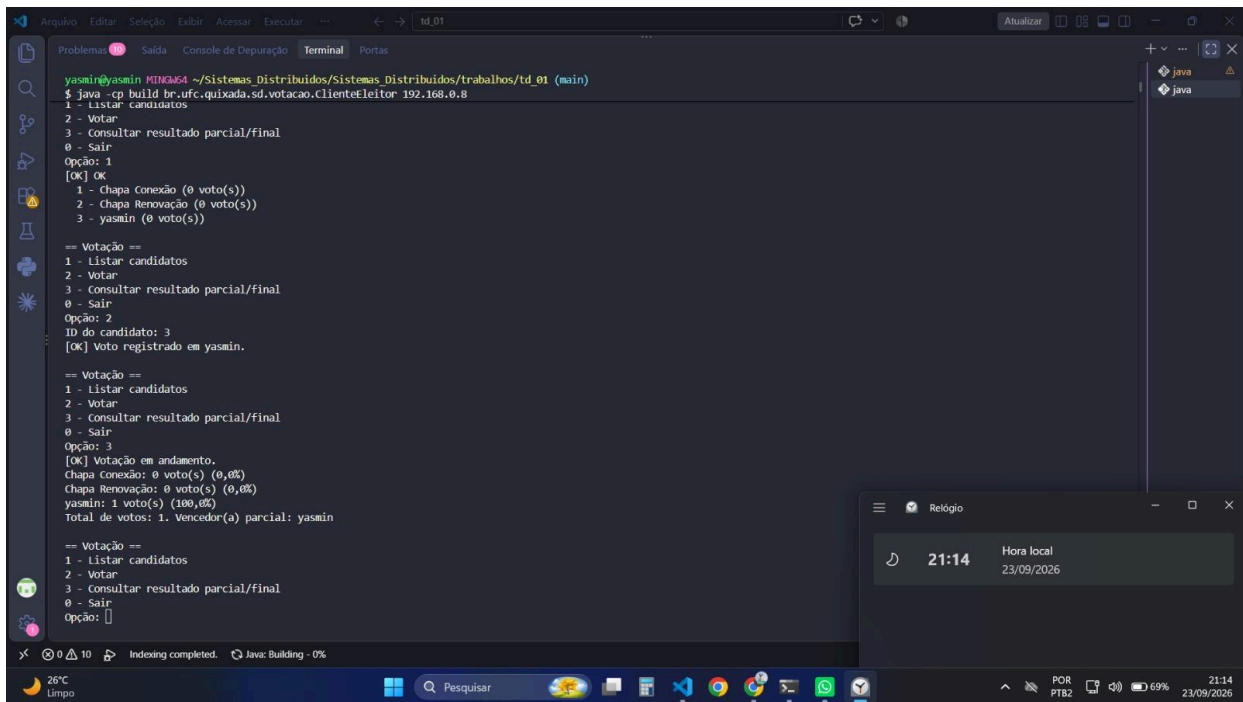


Figura 12 – Execução do cliente eleitor realizando autenticação e acesso às funcionalidades da votação.

```
yasmin@yasmin MINGW64 ~/Sistemas_Distribuidos/Sistemas_Distribuidos/trabalhos/td_01 (main)
$ java -cp build br.ufc.quixada.sd.votacao.ClienteAdmin 192.168.0.8
Senha de administrador: admin123
[OK] Login de administrador efetuado.

== Administração da votação ==
1 - Listar candidatos
2 - Adicionar candidato
3 - Remover candidato
4 - Enviar nota informativa (multicast)
5 - Consultar resultado parcial/final
0 - Sair
Opção: 2
Nome do candidato: yasmin
[OK] Candidato adicionado.
3 - yasmin (0 voto(s))

== Administração da votação ==
1 - Listar candidatos
2 - Adicionar candidato
3 - Remover candidato
4 - Enviar nota informativa (multicast)
5 - Consultar resultado parcial/final
0 - Sair
Opção: 4
Texto da nota: votacao em aberto
[OK] Nota informativa enviada a todos os eleitores.

== Administração da votação ==
1 - Listar candidatos
2 - Adicionar candidato
3 - Remover candidato
4 - Enviar nota informativa (multicast)
5 - Consultar resultado parcial/final
0 - Sair
Opção: 
```

Conclusão

Com a implementação, conseguimos colocar em prática os principais conceitos vistos na disciplina, como comunicação entre processos, troca de mensagens utilizando sockets, serialização de objetos, criação de streams e comunicação multicast. A atividade ajudou a entender melhor como funciona o desenvolvimento de uma aplicação distribuída, desde a comunicação entre cliente e servidor até os desafios de controlar diferentes processos executando ao mesmo tempo.